

A Programmer–Interpreter Neural Network Architecture for Prefrontal Cognitive Control

Francesco Donnarumma*

*Institute of Cognitive Sciences and Technologies
National Research Council of Italy
Via S. Martino della Battaglia 44-00185 Roma, Italy
francesco.donnarumma@istc.cnr.it*

Roberto Prevete

*Università degli Studi di Napoli Federico II
Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione (DIETI)
Via Claudio, 21, 80125 Napoli, Italy
roberto.prevete@unina.it*

Fabian Chersi

*University College London
Institute of Cognitive Neuroscience
17 Queen Square, London, WC1N 3AR, England
f.chersi@ucl.ac.uk*

Giovanni Pezzulo

*Institute of Cognitive Sciences and Technologies
National Research Council of Italy
Via S. Martino della Battaglia 44-00185 Rome, Italy
giovanni.pezzulo@istc.cnr.it*

Accepted 9 March 2015

Published Online 18 May 2015

There is wide consensus that the prefrontal cortex (PFC) is able to exert cognitive control on behavior by biasing processing toward task-relevant information and by modulating response selection. This idea is typically framed in terms of top-down influences within a cortical control hierarchy, where prefrontal-basal ganglia loops gate multiple input–output channels, which in turn can activate or sequence motor primitives expressed in (pre-)motor cortices. Here we advance a new hypothesis, based on the notion of *programmability* and an interpreter–programmer computational scheme, on how the PFC can flexibly bias the selection of sensorimotor patterns depending on internal goal and task contexts. In this approach, multiple elementary behaviors representing motor primitives are expressed by a single multi-purpose neural network, which is seen as a reusable area of “recycled” neurons (interpreter). The PFC thus acts as a “programmer” that, without modifying the network connectivity, feeds the interpreter networks with specific input parameters encoding the *programs* (corresponding to network structures) to be interpreted by the (pre-)motor areas. Our architecture is validated in a standard test for executive function: the 1-2-AX task. Our results show that this computational framework provides a robust, scalable and flexible scheme that can be iterated at different hierarchical layers, supporting the realization of multiple goals. We discuss the plausibility of the “programmer–interpreter” scheme to explain the functioning of prefrontal-(pre)motor cortical hierarchies.

Keywords: Programmable neural networks; pre-frontal cortex; cognitive control; computational model.

*Corresponding author.

1. Introduction

Current theories on executive function argue that prefrontal cortex (PFC) can flexibly bias the selection of sensorimotor patterns depending on internal goal and task contexts.^{46,55} This idea is typically framed in terms of top-down influences within a cortical control hierarchy.²⁵ However, the neural mechanisms underlying hierarchical control remain elusive. Firstly, it is unclear *what* is represented in the different layers; many proposals have been advanced that link PFC hierarchies to different levels of control, to increasingly more abstract information, or to events that are increasingly more remote in time.^{3,6,7,38,42} Secondly, it is not fully understood *how* higher hierarchical layers exert top-down influence or in other words what the computational principles regulating hierarchical control are, with competing proposals building on (hierarchical) reinforcement learning, dynamical systems, and predictive coding, among others.^{6,11,21,37,41,51,60,61,77}

Views of hierarchical brain functioning can broadly be distinguished into two main categories: localized or distributed. In the localist perspective, hierarchies can be composed of neural subpopulations having specialized functions (encoding e.g. behaviors or motor primitives) with higher hierarchical layers having the functional role to select and switch among them. In a computational model that is representative of this approach, prefrontal — basal ganglia loops *gate* multiple input channels^{51,67} or experts.^{21,35} Another view is to consider a single neural layer that can (learn to) encode multiple functions (behaviors) in a distributed fashion and can select a specific behavior on the basis of “control inputs” coming from higher hierarchical layers.^{8,43,79}

In this paper, we build upon a modeling proposal of flexible modular control¹⁵ that uses a distributed scheme and builds upon the concept of *programmability* as defined in computer science.⁷⁶ We developed a *programmer-interpreter* hierarchical architecture for cognitive control where a higher level “programmer” network (corresponding to PFC) selects inputs for a lower-level reusable neural network (corresponding to (pre-)motor cortex), thus

selecting among the multiple behaviors that the latter can potentially express.

To explain the properties of the proposed architecture, we compare it with a more standard, localist-modular approach in which each neural network encodes an “expert” separately. We test the two architectures in a well-known benchmark task requiring a hierarchical control structure: the 1-2-AX-continuous performance task, where subjects are asked to detect specific target sequences of letters and numbers.²² This comparison helps elucidating the strengths and weaknesses of the current proposal from a computational perspective. Furthermore, it clarifies the empirical predictions of the proposed scheme in terms of *what* is represented in the different layers of the hierarchical PFC — (pre-)motor circuit organization, and of *how* a higher hierarchical PFC layer exerts control over behavior by using control inputs to bias the functioning of the lower (pre-)motor layer.

2. Methods: Computational Approach

Traditional views of the brain as a modular system organized in local neural circuits specialized for single functions²⁰ are increasingly challenged by studies showing that the same neuronal structure can support many functions in different conditions^{2,4,24,54} and fulfill multiple demands.¹⁶ Based on such evidence, various authors such as Refs. 2 and 13 have proposed that low-level neural circuits are *re-cycled* (or *reused*) for various purposes by other neural circuits across different task categories and cognitive domains. Furthermore, many computational studies or brain theories are based on the idea that brain areas can be *controlled* by other parts of the brain and, consequently, perform different behaviors under different conditions.^{9,26,32,36,39,52,66,68,71} A new hypothesis gaining ground, on this ability of some areas to control other areas, is based on neural mechanisms that allow circuits to change their behaviors rapidly, dynamically, and reversibly, thus without changing their structure or changing (re-learning) synaptic connectivity.⁴

The view of the brain as a network of reusable areas that can be flexibly controlled by other areas

can potentially impact the way we conceive cognitive control and hierarchical brain function.¹² However, at the moment we still have few biologically realistic computational proposals explaining how the brain might implement the required “divergence of function from structure”⁵⁴: a fast and flexible re-assemblage and reuse of neuronal structure without changes in synaptic connectivity due to learning. Indeed, most current theories of cognitive control are framed within connectionist networks,⁵¹ where the standard paradigm to adaptation and control requires the modification of synaptic weights.⁶⁹ Despite so, the idea of controlling the behavior of an artificial neural network through its inputs while keeping its structure constant has been explored, too. For example, in Refs. 33, 34, 43, 57, 72 and 75 auxiliary inputs are able to modulate the network behavior. Among them the work in Ref. 30 has long been considered a plausible implementation of working memory function.⁵⁰ In other approaches (see, e.g. Refs. 15, 19, 29, 48, 49, 63, 69 and 73), the auxiliary input has the capacity of effectively “switching” the network between different behaviors in a rapid and reversible way; in particular, in Ref. 64 the role of “mixed selectivity” in subserving the cognitive functions ascribed to the PFC is analyzed across recording in neural activity of monkeys during an object sequence memory task and resulting in flexible and quick adaptation to execute new tasks.

Consequently, our proposal departs from the aforementioned approaches and takes as its starting point the *programmable neural network* (PNN) architecture introduced by Donnarumma *et al.*,¹⁵ which has two main peculiarities. Firstly, it is endowed with a *programming* capability (where programming has to be intended in the same sense as used in computer science), which permits to deeply control a sub-system’s behavior without changing connectivity and efficacies associated with the synaptic connections. Secondly, it follows a distributed representation scheme in which multiple motor primitives are embedded in the same neural population. A basic depiction of the PNN architecture is shown in Fig. 1. It is provided with two kinds of input lines: auxiliary (or programming) input lines and the standard data

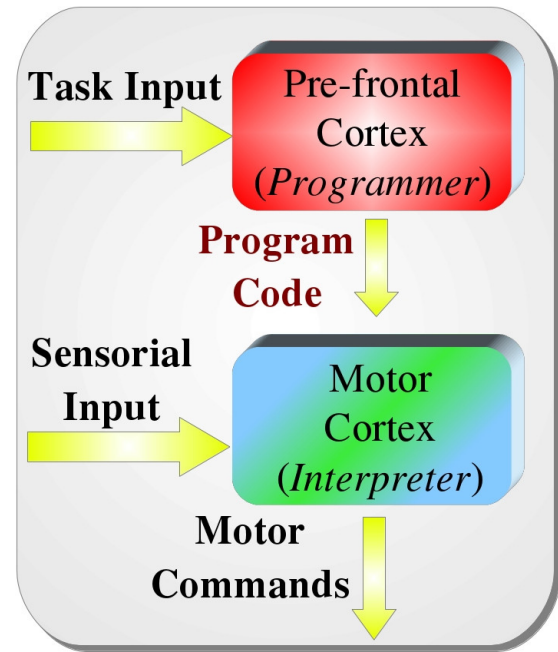


Fig. 1. Hierarchical two-layer architecture of the PNN architecture.¹⁵ In the proposed scheme, the top layer implements a neural programmer and corresponds to PFC function, and the bottom layer implements a neural interpreter, which might be implemented in motor and premotor areas of the brain.

input lines, in which the auxiliary input lines are fed with a code, or *program*, describing the weights of another network. Thus, one can develop a hierarchical two-layer architecture: a bottom neural layer processing auxiliary and standard input, and a top neural layer processing standard input and sending programs to the bottom layer by the auxiliary input lines. As shown in Fig. 1, these two layers implementing *programmer-interpreter* scheme might correspond to prefrontal and (pre-)motor brain areas, respectively.

The bottom layer is a neural network having both fixed connectivity and fixed synaptic efficacies which behaves as an *interpreter* able to simulate the neural network encoded in the auxiliary input (an action resembling feeding the code of a *virtual machine* into a standard computational architecture). The top layer can be described as a *programmer* which sends

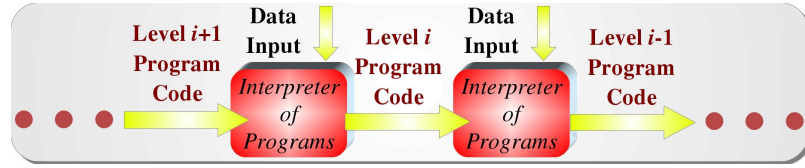


Fig. 2. Representation of the hierarchical coupling of program/interpreter modules allowed in a programmable neural architecture.

the code of a weight matrix to the bottom layer. This neural architecture allows to model “instantaneous” switches of behavior, which are required to implement the kind of rule-like function typical of cognitive control, without changing connectivity.

This computational scheme can easily be iterated at multiple hierarchical layers, with the result that a network playing the role of *programmer* relative to a lower-level *interpreter* can also play the role of an *interpreter* relative to a higher-level *programmer*, providing a homogeneous organizing principle for cortical hierarchies that extend over an indefinite number of layers, see Fig. 2.

At the computational level, this hierarchical architecture is composed of multiplicative subnetworks that enable a first (programmer) network to provide input values to a second (interpreter) network through auxiliary input lines. To clarify how this is possible, we recall that the dynamic behavior of an artificial neural network is read on an output y_i based on the sums of the products between connection weights w_{ij} and neuron output signals x_j :

$$y_i = f \left(\sum_j w_{ij} \cdot x_j \right).$$

From a mathematical point of view, one can “pull out” the multiplication operation $w_{ij} \cdot x_j$ by using a *multiplication* (mul) subnetwork that can compute

the result of the multiplication between the output and the weight, inputs to the mul subnetwork:

$$y_i = f \left(\sum_j \text{mul}(w_{ij}, x_j) \right).$$

This procedure (called *w-substitution* in Ref. 15) permits to build a PNN architecture where the weights become auxiliary inputs, see Fig. 3. As a consequence, a PNN is provided with two kinds of input lines: *auxiliary* (or *programming*) input lines and the *standard* data input lines. The newly introduced programming inputs are fed with a code, or program, describing the network to be “simulated”.

2.1. CTRNN implementation

In principle, a PNN architecture can be implemented using several kinds of recurrent neural networks. Here we introduce an implementation of PNN using Continuous Time Recurrent Neural Networks (CTRNNs), which are generally considered to be biologically plausible networks of neurons and are described by (1), see Refs. 5 and 31.

$$\tau_i \frac{dy_i}{dt} = -y_i + \sigma \left(\sum_{j=1}^N w_{ij} y_j + \theta_i + I_i^e(t) \right), \quad (1)$$

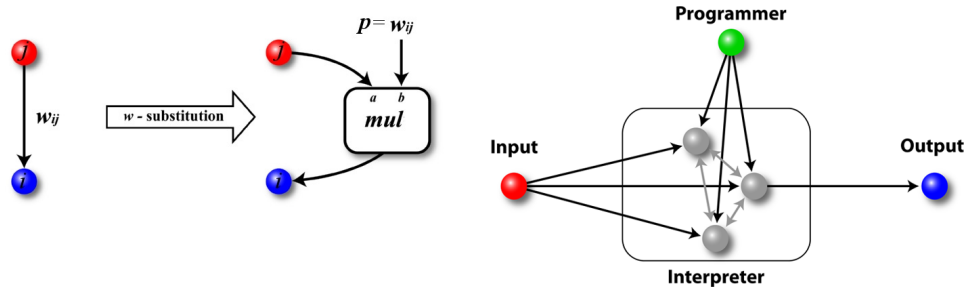


Fig. 3. The “pulling out” of the multiplication is depicted on top. By means of the procedure *w-substitution*, employing distinct mul networks, it is possible to effectively implement a PNN that acts as an interpreter of neural programs.

where $i \in \{1, \dots, N\}$ and N is the total number of the neurons in the network. Thus, for each neuron i : τ_i is the *membrane time constant*; y_i is the *mean firing rate*; θ_i is the *threshold* (or bias); $\sigma(x)$ is the standard logistic activation function, i.e. $\sigma(x) = \frac{1}{1+e^{-x}}$; $I_i^e = \sum_{j=N+1}^{N+L} w_{ij} x_j$ is an *external input current* coming from L external sources x_j ; finally, w_{ij} is the *synaptic efficacy (weight)* of the connection coming from the neuron j or external sources x_j to the neuron i .

The multiplication network mul is also implemented here in the CTRNN framework.^a The resulting PNN is a neural architecture with two kinds of input lines: data input lines and auxiliary (or programming) input lines. The programming input lines can be fed with codes (or programs) describing network structures in an effective way, and the PNN is able to simulate (behaving like an *interpreter*) the behavior of the encoded networks on the data coming from the standard input lines.

Consequently, we implemented the PFC area as a standard CTRNN network, see Eq. (2).

$$\begin{aligned} \tau_i^{\text{PFC}} \dot{y}_i^{\text{PFC}} = & -y_i^{\text{PFC}} \\ & + \sigma \left(\sum_{j=1}^P w_{ij}^{\text{PFC}} y_j^{\text{PFC}} + I_i^{\text{ext}} \right) \end{aligned} \quad (2)$$

with $i \in \{1, \dots, P\}$.

PFC receives sensorial inputs $I_i^{\text{ext}} = \sum_{l=1}^L S_{il} x_l^{\text{Sensor}}$. It is at the top of the hierarchy (see Fig. 4) and projects its connections on the (pre-)motor areas. Our PFC module is meant to fall into different attractor states “readable” on its neurons y_i^{PFC} , depending on the detected sensorial information arriving at this area. The value of these neurons are used as programming inputs sent to the (pre-)motor areas, which rapidly and reversibly change their behavior, when different inputs come from the PFC area. This is achieved in the PNN framework, implementing the (pre-)motor area as an *interpreter* of CTRNNs, by the equations expressed

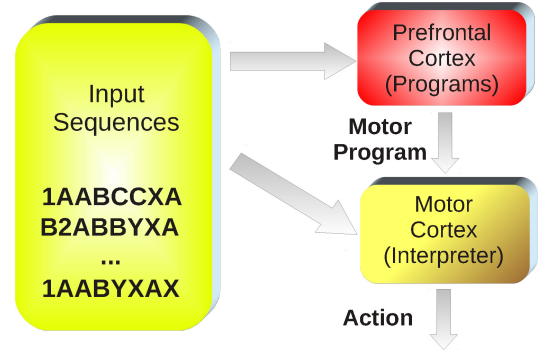


Fig. 4. The PNN architecture that solves the 1-2A-X task.

in (3).

$$\begin{cases} \tau_n \dot{y}_n = -y_n + \sigma \left(\sum_{h \in H_n} \hat{w} \cdot \mu_M^h + \sum_{j=1}^{N+L} \tilde{w} \cdot y_j \right) \\ \tau_m^k \dot{\mu}_m^k = -\mu_m^k + \sigma \left(\sum_{j=1}^N C_{mj} y_j \right. \\ \quad \left. + \sum_{j=1}^M C'_{mj} \mu_j^k + I_m^{\text{Program}} \right), \end{cases} \quad (3)$$

where

- $n \in \{1, \dots, N\}$,
- $y_{N+j} = x_j^{\text{Sensor}}$,
- $k \in \{1, \dots, N(N+L)\}$,
- $m \in \{1, \dots, M\}$,
- the time constants $\tau_m^k \ll \min\{\tau_n\}$, i.e. in a first approximation two dynamics are presents: the mul networks with faster dynamics, a subnetwork with a slower dynamics,
- μ_m^k is the activation of the m th neuron of the k th mul network,
- C_{mj} is a sparse matrix weighting connections from the slower subnetwork to the mul networks,
- C'_{mj} is the matrix of auto-connection of the mul networks,
- $\hat{w}$ weights connection from the mul networks to the slower subnetwork,

^aFor the sake of clarity, let us call here the implementation of the ideal mul as mul*. For technical reasons, mul* includes time constants $\tau^* \ll \tau_i$, where τ_i 's are the time constants of the network neurons not belonging to mul*. Indeed, in the implementation one might consider some “sources of noise” because (1) the mul* network has a finite time delay for approaching the asymptotic values, at variance with the zero delay of the ideal mul subnetworks introduced previously; (2) the asymptotic values $\bar{y}_{\text{mul}^*}$ could be not exactly the same as those provided by the ideal mul $\bar{y}_{\text{mul}}$, with a finite error $\delta = |\bar{y}_{\text{mul}^*} - \bar{y}_{\text{mul}}|$. See Ref. 15 for technical details.

- $\tilde{w}$ weights sensorial data,
- the term $I_m^{\text{Program}} = \sum_{j=1}^P C''_{mj} y_j^{\text{PFC}}$ constitutes the values of the programs, that are input signals sent from PFC weighted by the sparse C''_{mj} matrix.

We stress that in our modelization, all the connections of the (pre-)motor areas are *fixed* connections and thus, the dynamic behaviors that the (pre-)motor areas exhibit are due only to the change of its input, i.e. sensorial data (x_i^{Sensor}) and *programs* from PFC (y_j^{PFC}). This means that the changes of behavior we model in the (pre-)motor area are qualitatively different from learning, because not involving synaptic recalibration, and also *reversible*, because previous dynamic behaviors can be elicited whenever suitable programming inputs to the area are sent.

3. Experiments

In this section, we test our approach in the chosen 1-2-AX experimental task. Firstly, we clarify the particular setup and the scenario in Sec. 3.1, then in Sec. 3.2, we show the results obtained, which probe our hierarchical architecture to be a viable hypothesis of modeling how PFC actually solves a 1-2-AX task by “programming” (pre-)motor areas, i.e. selecting specific coding from a vocabulary of motor primitives.

3.1. Setup and procedure

We tested the behavior of the PNN architecture in the 1-2-AX-continuous performance task: this is a standard benchmark to evaluate executive function, and it has been widely used to study its candidate neuro-computational implementations.²² The task has an underlying hierarchical structure and has been designed to test the participants’ (or computational model’s) ability to selectively update the content of PFC working memory (here, the *program* to be used) and use it to select the most appropriate response. In the 1-2-AX task, letters and numbers are presented sequentially over time, and target sequences have to be detected. The appropriate response to a particular sequence (such as AX) is dependent on the most recently viewed number; thus, the cues (e.g. As) and probes (e.g. Xs) are nested hierarchically within an outer-loop signalled by the number information (e.g. 1s). In the 1-2-AX task

stimuli are presented one at a time in a sequence. In each subtask, an “outer loop” number (1 or 2) can switch at any moment the subtask to be executed (AX or BY). The participants’ (or model’s) task is to select one of two actions: *R* (right press) if the correct sequence is detected (say, AX preceded by 1) and *L* (left press) otherwise.

To make the task more challenging and interesting from a learning viewpoint, in addition to the two standard subtasks (1AX and 2BY) we added two more subtasks (3AX and 4BY). In this way, the problem cannot be solved by using fixed associations between 1A and 2B, but requires two nested loops: the former to set an “outer” context (1, 2, 3, or 4), and the latter to set an “inner” context (A or B), and the two contexts jointly contribute to determine the right response (X or Y). The alphabet used in our simulation is:

$$\Sigma = \{1, 2, 3, 4, A, B, X, Y\}.$$

The formal details of the desired subtasks are explained as regular expressions in Table 1. Sample example of the responses in correspondence of sample sequences are described in Fig. 5.

Figure 4 shows a depiction of the PNN architecture that solves the task: here the top (PFC) layer sets and maintains the program to be used (1AX, 2BY, 3AX, or 4BY) based on the current context (1, 2, 3, or 4) and feeds it to the bottom (pre-)motor layer, which then executes it to select the right motor response (*R* or *L*) based on the current stimulus (AX or BY). Note that both layers receive the input stream, but the top layer uses it to maintain or switch a program in working memory while the bottom layer uses it to select the motor response based on the current program received by the top layer.

The PNN implementation includes a hierarchy structured in a top layer and a bottom layer. The

Table 1. The regular expressions formally explaining the four subtasks implemented in the simulations. Sample sequences and responses are shown in Fig. 5.

Program name	Regular expression
1AX	$(\Sigma^*)1(\Sigma^*\{234\})AX$
2BY	$(\Sigma^*)2(\Sigma^*\{134\})BY$
3AX	$(\Sigma^*)3(\Sigma^*\{124\})A(\Sigma^*\{124\})X$
4BY	$(\Sigma^*)4(\Sigma^*\{234\})B(\Sigma^*\{234\})Y$

1	X	X	Y	A	X
1	X	Y	A	A	X
1	B	B	X	A	X
1	Y	B	Y	A	X
1	A	Y	X	A	X
1	X	A	Y	A	X
1	Y	A	X	B	Y

(a) Program 1AX

2	X	X	Y	B	Y
2	X	Y	A	B	Y
2	B	B	X	B	Y
2	Y	B	Y	B	Y
3	A	2	X	B	Y
2	X	A	Y	A	Y
2	Y	A	X	B	Y

(b) Program 2BY

3	X	X	Y	A	X
3	X	Y	A	A	X
3	X	A	B	Y	X
1	A	X	3	A	X
1	A	3	A	B	X
3	X	A	Y	A	X
3	A	Y	Y	B	X

(c) Program 3AX

4	A	A	Y	B	Y
2	B	Y	4	B	Y
4	X	B	B	X	Y
4	B	Y	3	B	Y
3	A	4	B	A	Y
4	X	B	Y	B	Y
4	B	A	X	B	Y

(d) Program 4BY

Fig. 5. (Color online) Sample sequences and responses for the different programs considered. Each row is a 6-character sequence. In Black are depicted Distractor stimuli (class D) related to the program; in Red the characters that elicit the correct program (class P), whose recognition is modeled in the PFC, and open the outer ‘Program’ Loop; in Blue the characters that open the inner ‘sequence recognition’ loop (class S); in Green, the characters that indicate the end of the recognition loop (class F) — at which the subject is required to respond or the Computational Model, in our case the outputs of our neural network architecture, is required to *fire*. In Panel (a), the example shows how the correct sequences accepted by Program 1AX are of the type PD^*SF , where D can be any character that does not cause the switching to another program, i.e. D can be any character in the alphabet Σ excluded 2, 3 and 4; the other characters are $P = 1$, $S = A$, $F = X$. In Panel (b) sample sequences for Program 2BY are shown. The sequences recognized are, analogously to the previous program, of the type PD^*SF , where in this case D can be any character in the alphabet Σ excluded 1, 3 and 4, $P = 2$, $S = B$, $F = Y$. Programs 3AX and 4BY respond to a different type of sequences, PD^*SD^*F , which admits a further loop between the character class S and F . Sample sequences of 3AX programs are shown in Panel (c), in which D can be any character in the alphabet Σ excluded 1, 2 and 4, $P = 3$, $S = A$, $F = X$ (notice that S and F are the same as in Program 1AX). Finally, in Panel (d), sample sequences are shown of Program 4BY. In this case, D can be any character in the alphabet Σ excluded 1, 2 and 3, $P = 4$, $R = B$, $F = Y$ (notice that S and F are the same as in Program 2BY). In Table 1 a formal definition of the sequences accepted are expressed as regular expressions.

top (*programmer*) layer of 14 neurons is meant to *detect and keep in memory* the four programs ($I = \{I_{1AX}, I_{2BY}, I_{3AX}, I_{4BY}\}$) and its implementations refer to Eq. (2). The bottom (*interpreter*) layer of 32 neurons acts as an interpreter of sequences and uses the inputs I provided by the programmer layer as well as the sensory input (e.g. 1AABCCXA) to output one of two motor commands $\{R, L\}$, represented here as two output neurons. The actual implementation of the bottom layer is realized by means of Eq. (3) (see also Fig. 7). It is important to stress that the bottom layer is a *fixed-structure* network (i.e. its weights are never changed during the simulations) and it changes its behavior only on the basis of the input I . This network can interpret even more programs without changing its structure (see the Discussion).

We compare the performance of this PNN network with an alternative, localist-modular scheme

(called MOD) in which the four different sub-tasks (i.e. recognize 1AX, 2BY, 3AX, or 4BY) are encoded in four smaller “compiled” CTRNNs of two neurons each. The MOD requires four different CTRNNs with a different structure of weights and four different outputs, which would correspond to separate and modular neuronal circuits in the brain. To work properly, MOD must include an extra component that collects the outputs of the separate networks and learns to arbitrate among them³⁵; because we use MOD as a reference to study the performance of PNN, here we assume that this extra module always arbitrates choice perfectly. We stress that Program 1AX and Program 2BY can be performed by identical MOD networks, as they perform the same program but with different input stimuli (see Fig. 5). The same applies with Program 3AX and 4BY: identical MOD networks can perform them too, for the same reasons. Thus, switching from 1AX

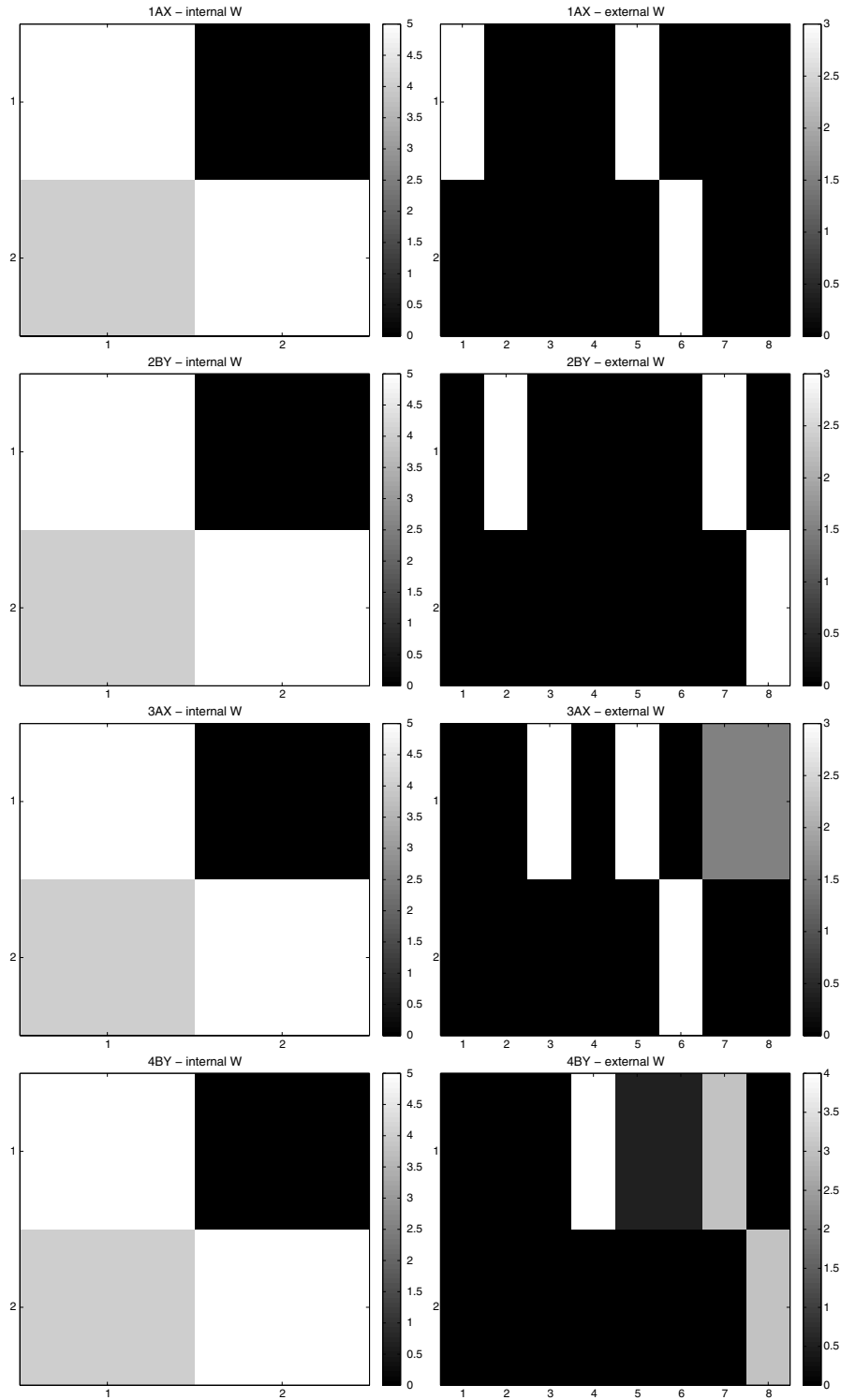


Fig. 6. The matrix strength connections are presented for each MOD network implementing the different programs 1AX, 2BY, 3AX, 4BY. The strength of the weights is depicted in gray scale. Each network has two neurons, thus, the internal matrix is a 2×2 matrix (on the left), while the external matrix is a 2×8 matrix, because the input signals coming from x_l^{Sensor} are $L = 8$.

to $2BY$ (respectively, from $3AX$ and $4BY$) can be thought only as a matter of suitable redirecting (*gating*) the proper input to the proper network. On the other hand, the switching among the other kinds of programs, say for instance the switching from $1AX$ to $3AX$, involves a structurally different kind of MOD networks, whose different functional working is not anymore a matter of mere gating. In those cases, in order to obtain a single network replicating the wanted MOD behaviors, it is crucial to resort to a PNN system: Such a system, in fact, by passing the corresponding *encoding* of a MOD network on the programming input line (i.e. by means of the *w*-substitution procedure), is proved to suitably *simulate* different MOD behaviors without changing its fixed synaptic structure.

3.2. Results

To test the models, we prepared a dataset composed of random sequences of symbols of the alphabet Σ . The tested sequences had 4000 elements, and there was a percentage of about 30% of the four different subtasks in it, with random distraction letters in the dataset. The total number of characters $L = 8$ is given at each time step as external sensorial input x_i^{Sensor} (see Sec. 2.1) by a 1-of- L encoding

scheme, i.e.:

		1	2	3	4	A	B	X	Y
		↓	↓	↓	↓	↓	↓	↓	↓
x_1^{Sensor}	→	1	0	0	0	0	0	0	0
x_2^{Sensor}	→	0	1	0	0	0	0	0	0
x_3^{Sensor}	→	0	0	1	0	0	0	0	0
x_4^{Sensor}	→	0	0	0	1	0	0	0	0
x_5^{Sensor}	→	0	0	0	0	1	0	0	0
x_6^{Sensor}	→	0	0	0	0	0	1	0	0
x_7^{Sensor}	→	0	0	0	0	0	0	1	0
x_8^{Sensor}	→	0	0	0	0	0	0	0	1

Figure 6 shows the strength of the connections of the different MOD networks implemented for each Program $1AX$, $2BY$, $3AX$, $4BY$. As it is possible to see, each module takes into account only inputs related to specific characters, putting to zero all the other distraction stimuli. Thus, one of the problems would be to correctly redirect the stimuli to the proper MOD area. Figure 7 shows the corresponding 1-2-AX interpreter built by means of the *w*-substitution procedure. It is possible to appreciate the organization principles in it. The matrix is very sparse and the network results organized in the substructures formally expressed in Eq. (2): a two-neuron network devoted to the readable output and 10 three-neuron-mul networks that interact with it. In particular, these mul networks modulate the

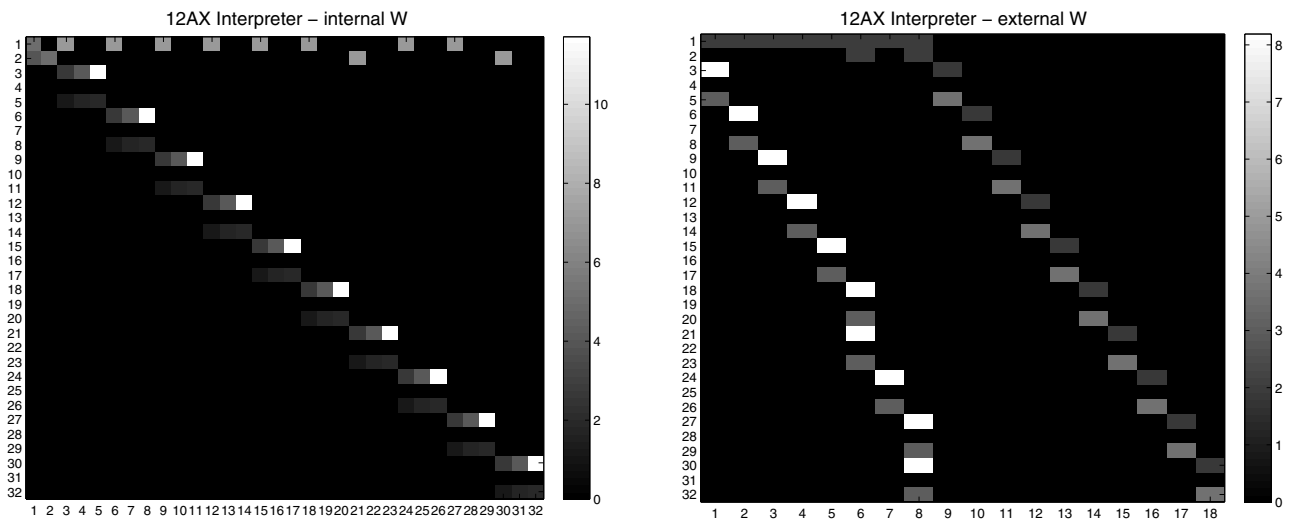


Fig. 7. The matrix strength connections are presented for the PNN interpreter implementing the 1-2-AX continuous performance task. The strength of the weights is depicted in gray scale. The network has 32 neurons, thus, the internal matrix is a 32×32 matrix (on the left). The external matrix receives from sensorial inputs x_i^{Sensor} (Channels 1–8) and from PFC module (Channels 9–18), resulting in a 32×18 matrix.

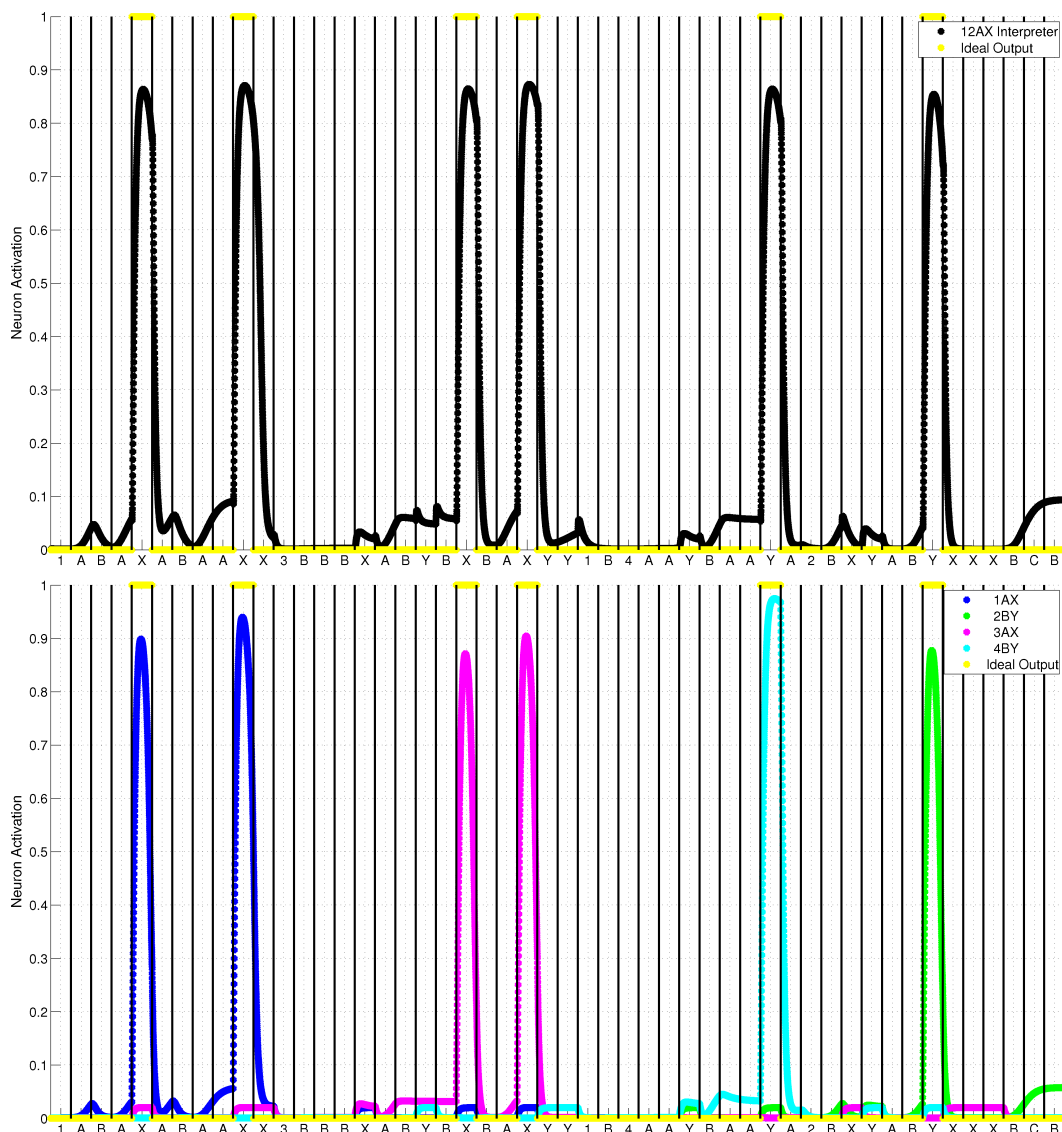


Fig. 8. Top: A sample execution of the PNN network is shown: the fixed-structure interpreter network fires in function of different characters of the sequence. Whenever a character in 1, 2, 3, 4, is presented, the network correctly *emulates* the proper program. Bottom: The MOD architecture constituted of four distinct networks (one for each sub-task) runs in parallel and fires separately whenever the proper subtask is presented.

programming input coming from PFC dynamically changing the output behavior in correspondence of PFC programs.

In our tests we initially presented each character with a fixed duration $T_{\max}$; then to test the robustness of the interpreter we gradually decrease length of the input character duration.

Figure 8 (top) shows a typical run of the PNN network. The programmer layer fires when responding to events in the “outer loop” and the interpreter layer recognizes the right subsequence in

a context-dependent way (that is, in a way that depends on which program it is inputted by the programmer layer). Figure 8 (bottom) shows the corresponding behavior of the MOD architecture, with four separate networks encoding one sub-task each.

The performance is evaluated varying the length of the signal choosing different percentages of the duration of the input symbols, $T_{\max}$. The percentage of $T_{\max}$ was varied in an interval $[83\%, 100\%]$. Figure 9 shows the accuracy of the reconstruction of the ideal target comparing the PNN and MOD

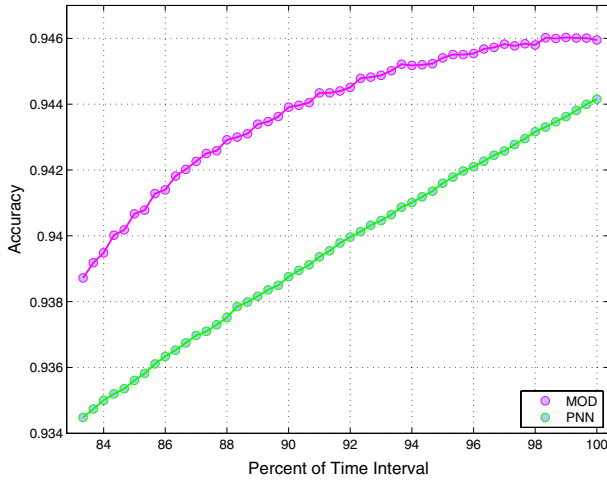


Fig. 9. (Color online) Accuracy of PNN (in magenta) versus MOD (in green) networks. The plot shows the accuracy of PNN (respectively, MOD) computed with respect to the target responses for different duration of input characters. See text for further details.

architectures, in function of the percentage of variation of $T_{\max}$ presented during the simulation. As it was expected the performance of MOD modules is higher when input signals duration increases. However, the slight degradation of PNN performance, is balanced by the acquisition of new functionalities, i.e. the possibility of recycling a neural structure in order to achieve multiple behaviors, ‘readable’ on the activation of the same output neurons.

Figure 10 shows the reconstruction error of the PNN interpreter while varying the duration of the input characters and while adding additional programs to the interpreter network. The increase of reconstruction error is due to the finite-time responding mul* networks, which limit the *interpreting* capability. In other words, the interpreter layer does not allow to simulate an infinite number of sequences with an infinite precision. Thus, a compromise is needed between the possibility of adding more behaviors and the noise resulting in the network representations.

These results permit to illustrate the trade-offs between the PNN strategy based on programming and recycled areas (i.e. using the bottom layer as a general-purpose interpreter) and the more standard strategy that modularizes and “compiles” the separate behaviors or programs using separate localist neural networks. An argument that is often used in favor of localist modules is that they permit to store an arbitrary number of elements without the so-called catastrophic forgetting,²³ while distributed schemes have problems in acquiring a high number of programs of behaviors.⁷⁹ Our results show that, not surprisingly, the performance of the PNN network decreases when more programs are added. However, this does not depend on catastrophic forgetting but on the aforementioned limits of the *interpreting* capability. Indeed, our framework differs from conventional approach with respect to learning because

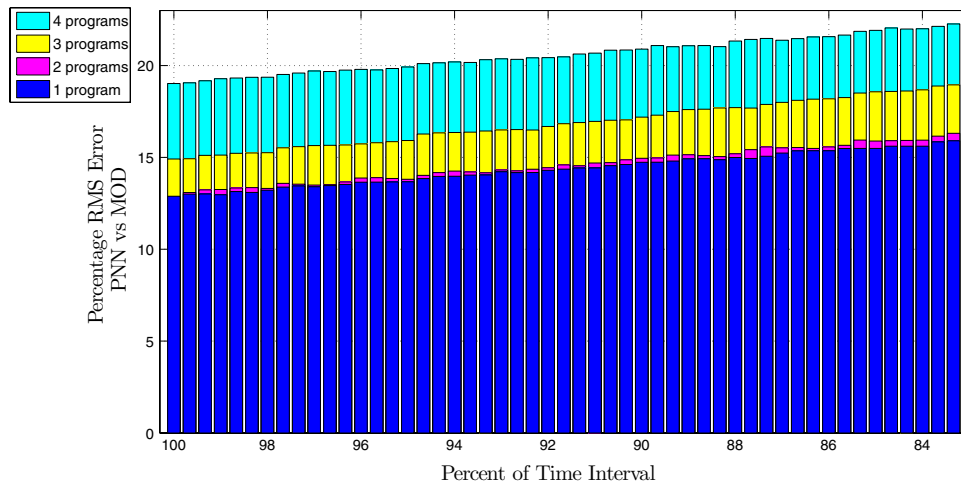


Fig. 10. The figure shows the reconstruction error of the PNN interpreter relative to one of the MOD modules, as a function of the duration of the input characters. The analysis is performed while adding one program at a time to the PNN interpreter and correspondingly one more module to MOD architecture.

learning a new behavior or a new element of information in a standard artificial neural network modifies the weights of the network, thus generally altering or erasing the previously acquired items (which causes catastrophic forgetting). In the interpreter-programmer organization the interpreter neural layer constitutes a fixed-weight network and the “learning” will have to be delegated to the appropriate selection of the programming inputs. Thus, while in conventional neural network architectures previous memories are consigned to the structure of the weights of the network, in our approach, the incremental acquisition of new behaviors or elements of information does not entail any structure modification, but only resorts to the acquisition of new programs. The limitation in the PNN architecture is thus more appropriately linked to a working memory function rather than to a learning function. The same limitation is also present in localist-modular schemes: although it is possible to encode multiple behaviors in separated modules, as the number of modules increases, arbitrating between them becomes problematic (note that in our examples we instead assumed perfect arbitration for the sake of simplicity). Overall, then, the proposed PNN scheme permits to encode multiple behaviors or programs; as in localist-modular schemes, the performance of the PNN architecture decreases when more programs are encoded but there is no catastrophic forgetting.

On the other hand, using a general-purpose interpreter as in the PNN architecture allows for a much faster acquisition of novel behaviors compared to the alternative strategies that require to learn a new module (in the localist-modular scheme) or to re-learn connections (in distributed connectionist approaches) whenever a new task is needed. In the PNN, given nets of N neurons with L inputs and a net mul of M neurons, the w -substitution lets us build an interpreter of at most $N + N \cdot M(N + L)$, larger than the single original networks. For w -substitution, each weight is replaced with a specific number of new connections. Supposing

that we want to build an interpreter of K different programs, we start the construction with K different network modules. This would result in a $K \cdot (N^2 + NL)$ total number of connections. After a complete w -substitution (made on all the connections) the resulting interpreter would have a number $M \cdot (N^2 + NL)$. Thus, when it occurs that $K > M$, it is more convenient, from the point of view of the connection size (i.e. network size), to construct a single interpreter of programs more than different networks each one specialized in a different behavior, as in the MOD architecture.^b Overall, then, these considerations point to a marked advantage of using recycled neural areas and, in particular, the PNN scheme.

4. Discussion

We proposed a hierarchical *interpreter-programmer* computational hypothesis for hierarchical brain function and cognitive control. The proposal builds on two main hypotheses: first, higher brain areas can “program” lower brain areas (i.e. control them using auxiliary inputs within a fixed-weight architecture), and second, the latter could include multiple behaviors using networks of recycled or reusable neurons. The hypothesis is implemented using a PNN architecture in Ref. 15. Here, the bottom (interpreter) network, putatively corresponding to (pre-)motor brain areas, can “store” multiple basic behaviors within a common neural substrate; this coding scheme is more parsimonious than alternative localist or distributed approaches, and also avoids the shortcomings of catastrophic forgetting. The top (programmer) network, putatively corresponding to prefrontal brain areas, can exert cognitive control and enforce rule-like behaviors by instantaneously instructing the interpreter network, without the necessity of re-learning.

Note that we exemplified the functioning of our PFC-(pre)motor model using a simplified, yet very widely adopted neural network architecture, the CTRNN. Although CTRNN-based systems

^bNote that this relationship only takes into account a perfect interaction between mul networks with no delays and noise. In an effective implementation also those factors should be taken into account and an empirical trade-off should be considered in order to prevent resulting simulating behaviors from being too noisy to be efficiently deployed in the desired task. Indeed, the need of an approximation for the mul* construction implies a degradation of the performance that should be carefully managed, although in our experiments the neural architecture exhibits a very good tolerance to the length of the input character duration.

introduce several simplifications compared, say, to spiking neuron network models,⁵⁶ they are commonly used to model real neural circuits (see, for example, Refs. 17, 47 and 78), and they are sufficient to highlight the main characteristics of our proposed architecture of brain organization, in particular, the ways brain areas at different hierarchical layers communicate. Importantly, it is now widely accepted that some brain areas with a fixed connectivity perform more than one behavior and that the transitions between the different behaviors occur in a rapid and reversible manner.^{4,54} Brain tissue endowed with a programming capability exhibits exactly this type of property.^{27,28} Programmable CTRNNs are a computational account of how *programmable* brain tissues can be obtained. However, implementing the proposed model of hierarchical brain organization using a more detailed neuronal architecture is certainly an important open point for future research.

We tested the PNN architecture in the 1-2-AX task, which is a typical benchmark for prefrontal function. Our results show that the PNN using a single interpreter compares well with a modular architecture that includes four “compiled” networks, each specialized for a single sub-task. The faster degradation of the performance of the PNN is compensated by its much more parsimonious neural structure in terms of the number of the connections required to encode programs or behaviors. Although the computational parsimony, robustness and scalability of this approach compared to alternative proposals are not per se sufficient to assess its biological plausibility, they at least provide computational-level arguments in favor of a possible advantage of the proposed scheme from an evolutionary viewpoint.

From a neuroscientific perspective, the proposed interpreter-programmer scheme is consistent with a role of PFC in biasing *instantaneously* behavior⁴⁶ without the necessity of re-learning; and in selecting among multiple basic behaviors that might be encoded in lower brain areas (e.g. pre-motor areas⁶⁵).

Intriguingly, the proposed scheme is consistent with very recent neuroscientific findings that draw attention to the existence of rapid and reversible changes of the “dynamics” of fixed neural structures underlying cognition and behavior.^{44,54,74} The current proposal also offers a mechanistic implementation of how networks of recycled¹³ or reused² neurons might encode multiple motor behaviors and

might be flexibly controlled by other brain areas in a multi-purpose way¹⁶ without changing synaptic connectivity. The auxiliary input lines (see Sec. 2) introduced in the model allowing this kind of control are “structurally” similar to contextual input lines used by other architectures insofar as they carry a further input with respect to standard inputs, however they differ from contextual input lines because they require stronger constraints on the connectivity organization (see Eq. (3)). As a consequence, this specific organization entails that the auxiliary input encodes the structure of a neural network which will be simulated by the interpreter receiving the auxiliary input rather than representing the context where a behavior should occur.^{14,15,18} This auxiliary input drives the lower-level (interpreter) area to express one among the multiple behaviors that it can simulate.

Besides, our proposal provides new insights into *what* is encoded by the hierarchical PFC organization, and *how* a hierarchical layer exerts control over behavior by biasing lower layers to execute specific functions by means of control inputs. In relation to the first (*what*) point, the activity of each neural layer of the hierarchy can be viewed as the code of a program to be sent to lower hierarchical levels. This scheme can be replicated at different hierarchical levels because a network playing the role of *programmer* relative to a lower-level *interpreter* can also play the role of an *interpreter* relative to a higher-level *programmer*. This idea provides a novel organizing principle for cortical hierarchies and their role in supporting goal-directed behaviors, and links well with information-theoretic accounts in which higher hierarchical levels encode more “distal” contextual information.³⁸ In this respect, it is important to note that the dynamics of the PNN architecture requires at least three different time scales: the multiplicative subnetwork time scale (very fast), the time scale of the “simulated” network (slower than the previous one), and, eventually, the time scale of any network that outputs codes to program other networks (still much slower). This view is coherent with the idea that the functional organization of the brain follows a “temporal hierarchy”.^{37,40,53,59,79}

In relation to the second (*how*) point, one hypothesis is that a specific neural mechanism (here, the mul subnetworks) might enable each layer to interpret the programs provided by higher areas and consequently

to switch rapidly and reversibly between different behaviors without changing the connectivity structure and the strength of the synapses. This mechanism would explain how PFC is able to rapidly and reversibly modify the functionality of (pre-)motor areas, letting them express different behaviors or sequences of behaviors, without necessarily storing these sequences. In other terms, in this scheme the activity of (pre-)motor circuits can be cast in terms of a *vocabulary of behaviors* that the PFC can use to dynamically select *sentences* or sequences of elements of the vocabulary.⁶²

Alternatively, programmability properties might be based on multiplicative responses at the dendritic or synaptic levels. There is a significant debate about whether, and to what extent, the multiplication might be performed by the synapses or the dendrites as attested by the vast literature on the nonlinear and specifically multiplicative response on some input signals.^{1,45,49,70} For example, the multiplication is thought to play a crucial role in coordinate transformation¹ or auditory processing.⁵⁸ Compared to a scheme in which small neuronal populations (the mul subnetworks) perform multiplications locally (that is, connection by connection), a neuronal architecture supporting multiplicative responses at the dendritic or synaptic levels would be simpler. Furthermore, the rapid and reversible changes required in the PNN model might be linked to extrasynaptic neuromodulators (see, for instance^{4,10,44}). The viability of these or alternative hypotheses remains to be empirically tested in future studies.

Acknowledgments

Research funded by the Human Frontier Science Program (HFSP), award number RGY0088/2014 and by the EU's FP7 under Grant agreement No. FP7-ICT-270108 (Goal-Leaders).

References

1. R. Andersen, L. Snyder, D. Bradley and J. Xing, Multimodal representation of space in the posterior parietal cortex and its use in planning movements, *Annu. Rev. Neurosci.* **20** (1997) 303–330.
2. M. L. Anderson, Neural re-use as a fundamental organizational principle of the brain — target article, *Behav. Brain Sci.* **33**(4) (2010) 245–266.
3. D. Badre, Cognitive control, hierarchy, and the rostro-caudal organization of the frontal lobes, *Trends Cogn. Sci.* **12** (2008) 193–200.
4. C. I. Bargmann, Beyond the connectome: How neuromodulators shape neural circuits, *Bioessays* (2012) 458–465.
5. R. D. Beer, On the dynamics of small continuous-time recurrent neural networks, *Adaptive Behav.* **3**(4) (1995) 469–509.
6. M. M. Botvinick, Hierarchical models of behavior and prefrontal function, *Trends Cogn. Sci.* **12** (2008) 201–208.
7. M. M. Botvinick, Y. Niv and A. Barto, Hierarchically organized behavior and its neural foundations: A reinforcement learning perspective, *Cognition* **119**(3) (2009) 262–280.
8. M. M. Botvinick and D. C. Plaut, Short-term memory for serial order: A recurrent neural network model, *Psychol. Rev.* **113**(2) (2006) 201–233.
9. F. Chersi, F. Donnarumma and G. Pezzulo, Mental imagery in the navigation domain: A computational model of sensory-motor simulation mechanisms, *Adaptive Behav.* **21**(4) (2013) 251–262.
10. S. Colici, O. Zelay and B. L. Bardakjian, Response neuromodulators based on artificial neural networks used to control seizure-like events in a computational model of epilepsy, *Int. J. Neural Syst.* **21**(5) (2011) 367–383.
11. F. Cona and M. Ursino, A multi-layer neural-mass model for learning sequences using theta/gamma oscillations, *Int. J. Neural Syst.* **23**(3) (2013) 1250036.
12. S. Dehaene, M. Kerszberg and J. P. Changeux, A neuronal model of a global workspace in effortful cognitive tasks, *Proc. Natl. Acad. Sci. U.S.A.* **95** (1998) 14529–14534.
13. S. Dehaene, Evolution of human cortical circuits for reading an arithmetic: The “neuronal recycling” hypothesis, in *From Monkey Brain to Human Brain: A Fyssen Foundation Symposium*, Chap. 8 (Bradford MIT Press, 2005), pp. 133–157.
14. F. Donnarumma, R. Prevete and G. Trautteur, How and over what timescales does neural reuse actually occur? Commentary on “neural re-use as a fundamental organizational principle of the brain”, by Michael L. Anderson, *Behav. Brain Sci.* **33**(4) (2010) 272–273.
15. F. Donnarumma, R. Prevete and G. Trautteur, Programming in the brain: A neural network theoretical framework, *Connect. Sci.* **24**(2–3) (2012) 71–90.
16. J. Duncan, The multiple-demand (md) system of the primate brain: Mental programs for intelligent behavior, *Trends Cogn. Sci.* **14**(4) (2010) 172–179.
17. N. A. Dunn, S. R. Lockery, J. T. Pierce-Shimomura and J. S. Conery, A neural network model of chemotaxis predicts functions of synaptic connections in the nematode *Caenorhabditis elegans*, *J. Comput. Neurosci.* **17**(2) (2004) 137–147.
18. C. Eliasmith, A unified approach to building and controlling spiking attractor networks, *Neural Comput.* **17**(6) (2005) 1276–1314.

19. C. Eliasmith and C. H. Anderson, *Neural Engineering: Computation, Representation, and Dynamics in Neurobiological Systems* (MIT Press, 2003).
20. J. A. Fodor, *Modularity of Mind: An Essay on Faculty Psychology* (MIT Press, Cambridge, MA, 1983).
21. M. J. Frank and D. Badre, Mechanisms of hierarchical reinforcement learning in corticostriatal circuits 1: Computational analysis, *Cereb. Cortex* **22** (2012) 509–526.
22. M. J. Frank, B. Loughry and R. C. O'Reilly, Interactions between frontal cortex and basal ganglia in working memory: A computational model, *Cogn. Affect. Behav. Neurosci.* **1**(2) (2001) 137–160.
23. R. M. French, Catastrophic forgetting in connectionist networks: Causes, consequences and solutions, *Trends Cogn. Sci.* **3**(4) (1999) 128–135.
24. J. Friedrich, R. Urbanczik and W. Senn, Code-specific learning rules improve action selection by populations of spiking neurons, *Int. J. Neural Syst.* **24**(5) (2014) 1450002.
25. J. M. Fuster, *The Prefrontal Cortex: Anatomy, Physiology, and Neuropsychology of the Frontal Lobe* (Lippincott-Raven, Philadelphia, PA, 1997).
26. M. Fyhn, T. Hafting, A. Treves, M.-B. Moser and E. I. Moser, Hippocampal remapping and grid realignment in entorhinal cortex, *Nature* **446** (2007) 190–194.
27. C. Garzillo and G. Trautteur, Computational virtuality in biological systems, *Theor. Comput. Sci.* **410**(4–5) (2009) 323–331.
28. C. D. Gilbert and M. Sigman, Brain states: Top-down influences in sensory processing, *Neuron* **54**(5) (2007) 677–696.
29. C. L. Giles, C. B. Miller, D. Chen, H. H. Chen, G. Z. Sun and Y. C. Lee, Learning and extracting finite state automata with second-order recurrent neural networks, *Neural Comput.* **4** (1992) 393–405.
30. S. Hochreiter and J. Schmidhuber, Long short-term memory, *Neural Comput.* **9** (1997) 1735–1780.
31. J. J. Hopfield and D. W. Tank, Computing with neural circuits: A model, *Science* **233** (1986) 625–633.
32. S. Hurley, The shared circuits model (SCM): How control, mirroring, and simulation can enable imitation, deliberation, and mindreading, *Behav. Brain Sci.* **31**(1) (2008) 1–22.
33. M. Ito and J. Tani, Generalization in learning multiple temporal patterns using RNNPB, *ICONIP: Int. Conf. Neural Information Processing* (2004), pp. 592–598.
34. M. I. Jordan, Attractor dynamics and parallelism in a connectionist sequential machine, in *Proc. Eighth Annual Conf. Cognitive Science Society* (Erlbaum, Hillsdale, NJ, 1986), pp. 531–546.
35. M. I. Jordan and R. A. Jacobs, Hierarchical mixtures of experts and the em algorithm, *Neural Comput.* **6**(2) (1994) 181–214.
36. S. Kalitzin, M. Koppert, G. Petkov and F. Lopes Da Silva, Multiple oscillatory states in models of collective neuronal dynamics, *Int. J. Neural Syst.* **24**(6) (2014) 1450020.
37. S. J. Kiebel, J. Daunizeau and K. J. Friston, A hierarchy of time-scales and the brain, *PLoS Comput. Biol.* **4** (2008) e1000209.
38. E. Koechlin and C. Summerfield, An information theoretical approach to prefrontal executive function, *Trends Cogn. Sci.* **11** (2007) 229–235.
39. M. Koppert, S. Kalitzin, D. Velis, F. Lopes Da Silva and M. A. Viergever, Dynamics of collective multistability in models of multi-unit neuronal systems, *Int. J. Neural Syst.* **24**(2) (2014) 1430004.
40. G. La Camera, A. Rauch, D. Thurbon, H.-R. Luscher, W. Senn and S. Fusi, Multiple time scales of temporal response in pyramidal and fast spiking cortical neurons, *J. Neurophysiol.* **96**(6) (2006) 3448–3464.
41. N. R. Luque, J. Garrido, R. Carrillo, S. Tolu and E. Ros, Adaptive cerebral spiking model embedded in the control loop: Context switching and robustness against noise, *Int. J. Neural Syst.* **21**(5) (2011) 385–401.
42. D. Maisto, F. Donnarumma and G. Pezzulo, Divide et impera: Subgoalting reduces the complexity of probabilistic inference and problem solving, *J. R. Soc. Interface* **12**(104) (2015) 1–13.
43. M. Maniadakis, P. Trahanias and J. Tani, Self-organizing high-order cognitive functions in artificial agents: Implications for possible prefrontal cortex mechanisms, *Neural Netw.* **33** (2012) 76–87.
44. E. Marder, Neuromodulation of neuronal circuits: Back to the future, *Neuron* **76**(1) (2012) 1–11.
45. B. W. Mel, Why have dendrites? A computational perspective, in *Dendrites*, eds. G. Stuart, N. Spruston and M. Häusser (Oxford University Press, 1999), pp. 271–289.
46. E. K. Miller and J. D. Cohen, An integrative theory of prefrontal cortex function, *Annu. Rev. Neurosci.* **24** (2001) 167–202.
47. P. Miller and X.-J. Wang, Inhibitory control by an integral feedback signal in prefrontal cortex: A model of discrimination between sequential stimuli, *Proc. Nat. Acad. Sci. U.S.A.* **103**(1) (2006) 201–206.
48. G. Montone, F. Donnarumma and R. Prevete, A robotic scenario for programmable fixed-weight neural networks exhibiting multiple behaviors, in *Adaptive and Natural Computing Algorithms*, eds. A. Dobnikar, U. Lotric and B. Šter, Lecture Notes in Computer Science, Vol. 6593 (Springer Berlin/ Heidelberg, 2011), pp. 250–259.
49. D. C. Noelle and G. W. Cottrell, Towards instructable connectionist systems, in *Computational Architectures Integrating Neural and Symbolic Processes*, eds. R. Sun and L. Bookman (Kluwer, 1995), pp. 187–221.

50. R. C. O'Reilly and M. J. Frank, Making working memory work: A computational model of learning in the prefrontal cortex and basal ganglia, *Neural Comput.* **18** (2006) 283–328.
51. R. C. O'Reilly, S. Herd and W. Pauli, Computational models of cognitive control, *Current Opin. Neurobiol.* **20**(2) (2010) 257–261.
52. E. Oztop and M. Arbib, Schema design and implementation of the grasp-related mirror neuron system, *Biol. Cybern.* **87**(2) (2002) 116–140.
53. R. W. Paine and J. Tani, Motor primitive and sequence self-organization in a hierarchical recurrent neural network, *Neural Netw.* **17**(8–9) (2004) 1291–1309.
54. H.-J. Park and K. Friston, Structural and functional brain networks: From connections to cognition, *Science* **342**(6158) (2013) 1238411.
55. R. E. Passingham and S. P. Wise, *The Neurobiology of the Prefrontal Cortex: Anatomy, Evolution, and the Origin of Insight* (Oxford University Press, Oxford, 2012).
56. H. Paugam-Moisy and S. Bohte, Computing with spiking neuron networks, in *Handbook of Natural Computing* (Springer, 2012), pp. 335–376.
57. A. Pedrocchi, S. Ferrante, E. De Momi and G. Ferrigno, Error mapping controller: A closed loop neuroprosthesis controlled by artificial neural networks, *J. Neuroeng. Rehab.* **3**(1) (2006) 25.
58. J. Pena and M. Konishi, Robustness of multiplicative processes in auditory spatial tuning, *J. Neurosci.* **24**(40) (2004) 8907–8910.
59. D. Perdikis, R. Huys and V. K. Jirsa, Time scale hierarchies in the functional organization of complex behaviors, *PLoS Comput. Biol.* **7** (2011) e1002198.
60. G. Pezzulo, An active inference view of cognitive control, *Front. Psychol.* **3**(478) (2012) 1–2.
61. G. Pezzulo, F. Donnarumma and H. Dindo, Human sensorimotor communication: A theory of signaling in online social interactions, *PLoS ONE* **8**(11) (2013) e79876.
62. G. Pezzulo, M. A. van der Meer, C. S. Lansink and C. M. Pennartz, Internally generated sequences in learning and executing goal-directed behavior, *Trends Cogn. Sci.* **18**(12) (2014) 647–657.
63. D. Prokhorov, L. Feldkarnp and I. Tyukin, Adaptive behavior with fixed weights in RNN: An overview, in *Proc. the 2002 Int. Joint Conf. Neural Networks, 2002. IJCNN '02*, Vol. 3 (2002), pp. 2018–2022.
64. M. Rigotti, O. Barak, M. R. Warden, X.-J. Wang, N. D. Daw, E. K. Miller and S. Fusi, The importance of mixed selectivity in complex cognitive tasks, *Nature* **497** (2013) 585–590.
65. G. Rizzolatti, R. Camarda, L. Fogassi, M. Gentilucci, G. Luppino and M. Matelli, Functional organization of inferior area 6 in the macaque monkey. ii. Area f5 and the control of distal movements, *Exp. Brain Res.* **71**(3) (1988) 491–507.
66. J. L. Rosselló, V. Canals, A. Oliver and A. Morro, Studying the role of synchronized and chaotic spiking neural ensembles in neural information processing, *Int. J. Neural Syst.* **24**(5) (2013) 1430003.
67. N. Rougier, D. Noelle, T. Braver, J. Cohen and R. O'Reilly, Prefrontal cortex and flexible cognitive control: Rules without symbols, *Proc. Natl. Acad. Sci. U.S.A.* **102**(20) (2005) 7338–7343.
68. A. Roy, Connectionism, controllers, and a brain theory, *IEEE Trans. Syst., Man Cybern. A, Syst. Hum.* **38** (2008) 1434–1441.
69. D. Rumelhart, G. Hinton and J. McClelland, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition* (MIT Press Cambridge, MA, USA, 1986), pp. 605–636.
70. E. Salinas and L. Abbott, A model of multiplicative neural responses in parietal cortex, *Proc. Natl. Acad. Sci. U.S.A.* **93**(21) (1996) 11956–11961.
71. E. Sauser and A. Billard, Parallel and distributed neural models of the ideomotor principle: An investigation of imitative cortical pathways, *Neural Netw.* **19**(3) (2006) 285–298.
72. J. Schmidhuber, Learning to control fast-weight memories: An alternative to dynamic recurrent networks, *Neural Comput.* **4**(1) (1992) 131–139.
73. H. T. Siegelmann, *Neural Networks and Analog Computation Beyond the Turing Limit* (Birkhäuser, 1999).
74. D. Sussillo, Neural circuits as computational dynamical systems, *Curr. Opin. Neurobiol.* **25** (2014) 156–163.
75. D. S. Touretzky, Boltzcons: Dynamic symbol structures in a connectionist network, *Artif. Intell.* **46**(1–2) (1990) 5–46.
76. A. M. Turing, On computable numbers, with an application to the entscheidungsproblem, *Proc. Lond. Math. Soc.* **42**(2) (1937) 230–265.
77. M. Versace and M. Zorzi, The role of dopamine in the maintenance of working memory in prefrontal cortex neurons: Input-driven versus internally-driven networks, *Int. J. Neural Syst.* **20**(4) (2010) 249–265.
78. J.-X. Xu and X. Deng, Biological modeling of complex chemotaxis behaviors for C. elegans under speed regulations dynamic neural networks approach, *J. Comput. Neurosci.* **35**(1) (2013) 19–37.
79. Y. Yamashita and J. Tani, Emergence of functional hierarchy in a multiple timescale neural network model: A humanoid robot experiment, *PLoS Comput. Biol.* **4**(11) (2008) e1000220.